

UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA

Simulacija distribuiranog konsenzusa primenom NBFT algoritma

Dokumentacija

Student:

Jelena Gajić

E2 16/2025

Novi Sad, 2026.

Sažetak

Ova dokumentacija opisuje projektovanje, implementaciju i proveru simulatora distribuiranog konsenzusa zasnovanog na algoritmu NBFT iz rada *Improved Fault-Tolerant Consensus Based on the PBFT Algorithm*. Projekat je realizovan u programskom jeziku C++. Svaki simulirani čvor predstavlja nezavisnu izvršnu jedinicu sa sopstvenom niti, redom dolaznih poruka i lokalnim stanjem. Komunikacija se obavlja isključivo razmenom objekata tipa **Message**, dok klasa **Network** formira topologiju, bira glavnog čvora i predstavnike grupa, isporučuje poruke i prikuplja odgovore klijenta.

Implementirano je konzistentno heširanje za izbor uloga, podela čvorova u grupe, šest glavnih faza NBFT protokola, lokalno formiranje sertifikata, node-decision broadcast mehanizam, threshold vote-counting i klijentska potvrda konsenzusa. Simulator podržava normalno izvršavanje i tri kategorije grešaka: slanje pogrešne vrednosti, slanje nedovoljnog broja potpisa od strane predstavnika grupe i neodgovaranje primarnog čvora. Izvršavanje se prati preko numerisanih događaja na standardnom izlazu.

Dokumentacija daje teorijsku osnovu algoritma, precizno mapiranje zahteva projekta na programske komponente, opis svih datoteka i važnih funkcija, tok svake protokolske faze, način sinhronizacije niti, scenarije testiranja, rezultate kontrolnog pokretanja, uočena pojednostavljenja i predloge za dalje unapređenje.

Sadržaj

Sažetak	i
1 Uvod	1
1.1 Problem distribuiranog konsenzusa	1
1.2 Motivacija za NBFT	1
1.3 Cilj i obim projekta	2
2 Zahtevi projekta i njihova realizacija	3
3 Teorijska osnova NBFT algoritma	4
3.1 Uloge učesnika	4
3.2 Parametri sistema	4
3.3 Konzistentno heširanje i izbor uloga	6
3.4 Šest glavnih faza protokola	6
3.4.1 CLIENT_REQUEST	7
3.4.2 PRE_PREPARE1	7
3.4.3 IN_PREPARE1	7
3.4.4 IN_PREPARE2	7
3.4.5 OUT_PREPARE	8
3.4.6 COMMIT	8
3.4.7 PRE_PREPARE2 i REPLY	8
3.5 Node-decision broadcast model	8
3.6 Threshold vote-counting model	9
3.7 Teorijska komunikaciona složenost	9
4 Arhitektura simulatora	11
4.1 Pregled komponenti	11
4.2 Procesni model	12
4.3 Životni ciklus simulacije	12
5 Model poruke – Message.hpp	13

5.1	Tipovi poruka	13
5.2	Polja strukture Message	13
5.3	Razlog korišćenja skupa potpisa	14
6	Klasa Network	15
6.1	Odgovornosti klase	15
6.2	Konstruktor i validacija parametara	15
6.3	Inicijalizacija topologije	16
6.4	Formiranje heš-prstena	16
6.5	Dostava poruka	17
6.6	Vrste broadcast-a	17
6.7	Klijentska odluka	17
6.8	Logovanje	18
7	Klasa Node	19
7.1	Lokalno stanje čvora	19
7.2	Faze lokalnog automata	19
7.3	Konkurentna petlja	20
7.4	Dispatcher poruka	21
7.5	Obrada pojedinačnih faza	21
7.5.1	handleClientRequest	21
7.5.2	handlePrePrepare1	21
7.5.3	handleInPrepare1	21
7.5.4	handleInPrepare2	21
7.5.5	sendNormalOutPrepare	21
7.5.6	sendNodeDecision	22
7.5.7	handleOutPrepare	22
7.5.8	trySendCommit	22
7.5.9	handleCommit	22
7.5.10	handlePrePrepare2 i decideAndReply	22
8	Scenariji grešaka	23
8.1	FaultMode	23
8.2	Konfigurabilni scenariji	23
8.2.1	normal	23
8.2.2	byzantine_wrong_value	23
8.2.3	faulty_rep_low	24
8.2.4	primary_silent	24
8.3	Izbor neispravnih čvorova	24
9	Pokretanje i korišćenje	25

9.1	Struktura datoteka	25
9.2	Kompajliranje	25
9.3	Argumenti komandne linije	25
9.4	Validacija ulaza	26
10	Testiranje i rezultati	27
10.1	Metod kontrolne provere	27
10.2	Rezime ishoda	27
10.3	Broj poslatih poruka	28
10.3.1	Zašto normalni tok ima 98 poruka	28
10.4	Analiza scenarija sa pogrešnom vrednošću	29
10.5	Analiza neispravnog predstavnika	29
10.6	Primer završnog stanja	29
11	Bezbednosne provere u implementaciji	30
11.1	Provere pošiljaoca i uloge	30
11.2	Provere potpisa	30
11.3	Sprečavanje višestrukog brojanja	31
11.4	Razdvajanje glasova po vrednosti	31
12	Pojednostavljenja i ograničenja	32
12.1	Simulirani, a ne kriptografski potpisi	32
12.2	Jedan proces i memorijska komunikacija	32
12.3	Statičan view	32
12.4	Jedna konsenzusna instanca	32
12.5	Nepotpuna timeout logika node-decision modela	33
12.6	Pojednostavljen dokaz COMMIT poruke	33
12.7	Nema eksplicitne detekcije ekvivokacije	33
12.8	Log samo na standardnom izlazu	33
13	Predlozi za dalje unapređenje	34
14	Zaključak	36
A	Pregled javnih metoda	37
A.1	Network	37
A.2	Node	37
B	Pseudokod kompletne simulacije	39
C	Kontrolna lista za demonstraciju	41

Spisak slika

3.1	Princip izbora čvora na konzistentnom heš-prstenu	6
3.2	Logički tok poruka u simulatoru	7
4.1	Arhitektura simulatora u jednom procesu	11
7.1	Pojednostavljen automat faza čvora	20

Spisak tabela

2.1	Mapiranje zahteva specifikacije na implementaciju	3
3.1	Parametri tipične konfiguracije simulatora	5
5.1	Tipovi poruka	13
5.2	Polja poruke u trenutnoj implementaciji	14
6.1	Broadcast metode klase Network	17
8.1	Režimi ponašanja čvora	23
10.1	Rezultati kontrolnog pokretanja	27
10.2	Broj SEND događaja po scenariju	28

Poglavlje 1

Uvod

1.1 Problem distribuiranog konsenzusa

Distribuirani sistem se sastoji od više nezavisnih učesnika koji komuniciraju preko mreže i nemaju zajedničku memoriju niti centralni sat. Kada takvi učesnici treba da usvoje istu vrednost, redosled događaja ili novo stanje sistema, potreban je algoritam konsenzusa. Primer dogovorene vrednosti može biti blok transakcija u blockchain mreži, rezultat distribuiranog izračunavanja ili komanda koju više replika treba jednako da izvrši.

Korektan algoritam konsenzusa treba da obezbedi najmanje dve osnovne osobine:

- **bezbednost (safety)** – dva ispravna čvora ne smeju doneti različite odluke za istu instancu konsenzusa;
- **živost (liveness)** – pod odgovarajućim pretpostavkama o mreži i broju neispravnih čvorova, ispravni čvorovi treba konačno da donesu odluku.

Poseban problem predstavljaju vizantijske greške. Vizantijski čvor ne mora samo da prestane sa radom, već može slati različite poruke različitim primaocima, glasati za pogrešnu vrednost, lažno predstavljati dokaze ili pokušati da blokira napredovanje protokola. Klasičan model vizantijskog konsenzusa potiče iz problema vizantijskih generala [3].

1.2 Motivacija za NBFT

Praktična vizantijska tolerancija grešaka, odnosno PBFT, omogućava dogovor u prisustvu vizantijskih čvorova, ali klasičan tok zahteva intenzivnu komunikaciju između velikog broja replika [2]. Kada broj čvorova raste, broj poruka raste približno kvadratno, zbog čega protokol postaje težak za primenu u velikim mrežama.

NBFT pristup pokušava da smanji komunikacioni trošak tako što čvorove deli u manje grupe. Najpre se postiže lokalni dogovor unutar svake grupe, a zatim predstavnici grupa učestvuju u međugrupnom delu protokola. Da neispravan predstavnik ne bi imao potpunu kontrolu nad svojom grupom, rad uvodi node-decision broadcast model. Da bi se ograničio uticaj lažnih glasova, uvodi se threshold vote-counting model [1].

1.3 Cilj i obim projekta

Cilj projekta je izrada razumljivog simulatora koji prikazuje kako NBFT protokol funkcioniše u normalnim i neispravnim uslovima. Implementacija nije puna blockchain platforma niti kriptografski sistem. Ona je obrazovni model koji čuva najvažnije odnose između uloga, faza, pragova glasanja i poruka.

Konkretno, projekat obuhvata:

- simulaciju više čvorova sa sopstvenim stanjem;
- nezavisnu `std::thread` nit za svaki čvor;
- red poruka, mutex i uslovnu promenljivu po čvoru;
- izbor primary čvora, podelu u grupe i izbor predstavnika pomoću 32-bitnog heša;
- šest NBFT faza: `PRE_PREPARE1`, `IN_PREPARE1`, `IN_PREPARE2`, `OUT_PREPARE`, `COMMIT` i `PRE_PREPARE2`;
- simulirane potpise kao skup identifikatora čvorova;
- lokalni i globalni prag, kao i prag odgovora klijentu;
- node-decision broadcast kada predstavnik pošalje neispravan lokalni sertifikat;
- konfigurabilne scenarije grešaka i detaljan konzolni log.

Poglavlje 2

Zahtevi projekta i njihova realizacija

Projektna specifikacija zahteva distribuirane čvorove, konkurentno izvršavanje, komunikaciju porukama, implementaciju pravila dodeljenog algoritma, simulaciju grešaka i prikaz toka izvršavanja. Tabela 2.1 pokazuje kako je svaki zahtev realizovan u trenutnoj verziji projekta.

Tabela 2.1: Mapiranje zahteva specifikacije na implementaciju

Br.	Zahtev	Realizacija u projektu
1	Distribuirani čvorovi	Klasa <code>Node</code> čuva identitet, grupu, ulogu, fazu, prihvaćenu vrednost, glasove, sertifikate i odluku. Svaki objekat sam obrađuje poruke.
2	Konkurentno izvršavanje	Svaki <code>Node</code> pokreće sopstveni objekat <code>std::thread</code> . Čekanje poruka se vrši preko <code>std::condition_variable</code> .
3	Komunikacija	Poruke su objekti strukture <code>Message</code> . Klasa <code>Network</code> podržava unicast i više vrsta broadcast-a.
4	Pravila algoritma	Implementirane su NBFT faze, grupisanje, predstavnici grupa, lokalni kvorum, node-decision i threshold vote-counting.
5	Simulacija grešaka	Dostupni su režimi <code>WRONG_VALUE</code> , <code>LOW_SIGNATURES</code> i <code>SILENT</code> , izloženi kroz četiri scenarija komandne linije.
6	Prikaz izvršavanja	Metoda <code>Network::log</code> numeriče svaki događaj, prikazuje slanje poruka, promene faza, formiranje dokaza i konačno stanje.

Poglavlje 3

Teorijska osnova NBFT algoritma

3.1 Uloge učesnika

U jednoj instanci konsenzusa postoje sledeće uloge:

Klijent

šalje zahtev primarnom čvoru i na kraju prikuplja odgovore replika.

Primary čvor

inicira protokol, šalje prvu globalnu poruku, prihvata međugrupni dokaz i šalje završnu globalnu poruku.

Običan čvor grupe

proverava predlog, šalje lokalni glas predstavniku, proverava sertifikat i po potrebi zaobilazi neispravnog predstavnika.

Predstavnik grupe

agregira lokalne glasove, formira lokalni sertifikat, razmenjuje OUT_PREPARE poruke sa drugim predstavnicima i može poslati COMMIT primary čvoru.

Primary nije član nijedne grupe u implementaciji. Preostalih $n - 1$ čvorova raspoređuje se u R grupa veličine m .

3.2 Parametri sistema

Neka je:

$$n - \text{--ukupan broj čvorova, uključujući primary,} \quad (3.1)$$

$$m - \text{--broj čvorova u jednoj grupi,} \quad (3.2)$$

$$R = \left\lfloor \frac{n-1}{m} \right\rfloor, \quad (3.3)$$

$$E = \left\lfloor \frac{m-1}{3} \right\rfloor, \quad (3.4)$$

$$w = \left\lfloor \frac{R-1}{3} \right\rfloor. \quad (3.5)$$

U kodu se zahteva da važi:

$$m = 3E + 1, \quad (3.6)$$

kao i da se svih $n - 1$ replika može potpuno rasporediti u grupe:

$$n = Rm + 1. \quad (3.7)$$

Dodatno, rad pretpostavlja $R \geq 4$, pa konstruktor odbacuje manje konfiguracije.

Tri ključna praga u simulatoru su:

$$Q_{local} = 2E + 1, \quad (3.8)$$

$$H_{global} = (R - w)m, \quad (3.9)$$

$$Q_{reply} = \left\lfloor \frac{n-1}{2} \right\rfloor + 1. \quad (3.10)$$

Za tipičnu konfiguraciju $n = 17$ i $m = 4$ važi:

Tabela 3.1: Parametri tipične konfiguracije simulatora

Veličina	Formula	Vrednost
Broj grupa	$R = (17 - 1)/4$	4
Lokalna tolerancija	$E = (4 - 1)/3$	1
Tolerisane abnormalne grupe	$w = (4 - 1)/3$	1
Lokalni kvorum	$2E + 1$	3
Globalni prag	$(R - w)m$	12
Klijentski prag	$(n - 1)/2 + 1$	9

3.3 Konzistentno heširanje i izbor uloga

Rad predlaže konzistentno heširanje kako uloge ne bi bile unapred fiksirane i lako predvidive [4, 1]. U simulatoru se svaki čvor mapira na 32-bitni heš prstena na osnovu simulirane IP adrese:

$$h_i = \text{hash}(10.0.0.(i + 1)). \quad (3.11)$$

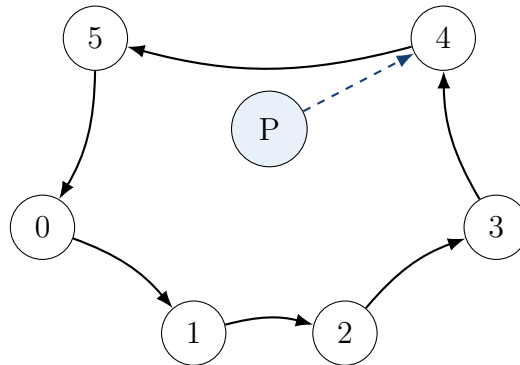
Funkcija `hashText` implementira FNV-1a stil 32-bitnog heširanja. Čvorovi se sortiraju rastuće po hešu i formiraju kružni poredak. Ako ciljni heš ne pronađe čvor sa većim ili jednakim hešem, izbor se vraća na prvi element prstena.

Primary se bira pomoću cilja:

$$h_P = \text{hash}(\text{masterIp} + \text{previousHash} + \text{view}). \quad (3.12)$$

Izabrani primary je prvi čvor u smeru kazaljke na satu čiji je heš veći ili jednak h_P . Nakon uklanjanja primary čvora, ostali čvorovi se redom dele u grupe od po m članova. Za svaku grupu g pravi se lokalni heš-prsten i bira predstavnik korišćenjem:

$$h_{rep,g} = \text{hash}(\text{masterIp} + \text{view} + g). \quad (3.13)$$

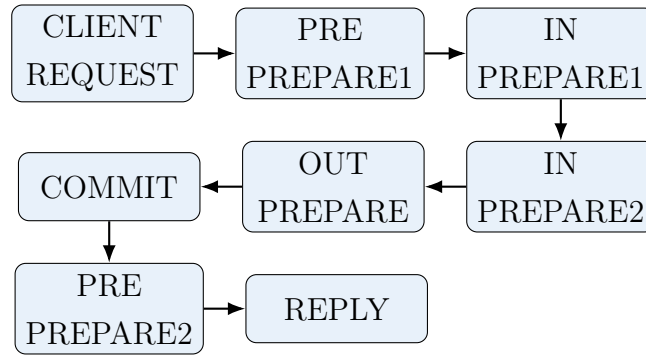


Pojednostavljen prikaz heš-prstena; izbor se vrši prvim čvorom u smeru kazaljke na satu.

Slika 3.1: Princip izbora čvora na konzistentnom heš-prstenu

3.4 Šest glavnih faza protokola

Tok algoritma iz rada prikazan je kroz šest faza. Klijentski zahtev i završni odgovor nisu deo tih šest faza, ali su neophodni za kompletan tok simulacije.



Slika 3.2: Logički tok poruka u simulatoru

3.4.1 CLIENT_REQUEST

Klijent šalje vrednost primary čvoru. Poruka nosi trenutni `view`, redni broj instance `sequenceNumber` i vrednost, koja je u demonstraciji `BLOCK_1`. Ako primary nije tih, prihvata zahtev i pokreće protokol.

3.4.2 PRE_PREPARE1

Primary šalje predloženu vrednost svim ostalim čvorovima. Replika prihvata poruku samo ako dolazi od trenutno izabranog primary čvora. Svaka replika pamti vrednost i prelazi u fazu `IN_PREPARE1`.

3.4.3 IN_PREPARE1

Svaki čvor šalje glas predstavniku svoje grupe. Glas sadrži tačno jedan simulirani potpis – identifikator pošiljaoca. Vizantijski čvor u režimu `WRONG_VALUE` umesto originalne vrednosti šalje niz sa dodatkom `_WRONG`.

Predstavnik vodi mapu:

$$\text{value} \longrightarrow \{\text{voterId}\}, \quad (3.14)$$

tako da se glasovi za različite vrednosti ne mešaju. Kada za jednu vrednost primi najmanje $2E + 1$ različitih glasova, formira lokalni sertifikat.

3.4.4 IN_PREPARE2

Predstavnik šalje lokalni sertifikat ostalim članovima svoje grupe. Običan ispravan čvor proverava:

1. da je poruku poslao predstavnik njegove grupe;

2. da se vrednost poklapa sa ranije prihvaćenim predlogom;
3. da postoji najmanje $2E + 1$ potpisa;
4. da svaki potpis pripada članu iste grupe.

Ako bilo koja provera ne uspe, čvor koristi node-decision broadcast.

3.4.5 OUT_PREPARE

U normalnom toku predstavnik šalje lokalni sertifikat svim predstavnicima grupa. Poruka nosi broj potpisa F i izračunatu težinu P . U abnormalnom toku običan čvor šalje sopstveni glas predstavnicima drugih grupa i postavlja `nodeDecision=true`.

3.4.6 COMMIT

Svaki ispravan predstavnik sakuplja doprinose grupa. Kada zbir validnih težina H dostigne globalni prag $(R - w)m$, šalje primary čvoru **COMMIT** poruku. U trenutnoj implementaciji više predstavnika može skoro istovremeno poslati **COMMIT**; primary prihvata samo prvi validan dokaz jer zastavica `finalBroadcastSent_` sprečava ponavljanje završnog broadcast-a.

3.4.7 PRE_PREPARE2 i REPLY

Primary šalje završnu vrednost svim replikama i uz poruku prosleđuje globalnu težinu. Ispravan čvor prihvata poruku samo ako ona dolazi od primary čvora, ako je težina dovoljna i ako se vrednost ne sukobljava sa njegovom ranije prihvaćenom vrednošću. Nakon prihvatanja prelazi u stanje **DECIDED** i šalje klijentu **REPLY**.

Klijent broji različite pošiljaoce za svaku vrednost zasebno. Konsenzus potvrđuje kada broj odgovora za istu vrednost dostigne Q_{reply} .

3.5 Node-decision broadcast model

Node-decision model je zaštita od neispravnog predstavnika grupe. U radu postoje tri razloga za aktiviranje: različita vrednost, premalo potpisa i istek vremena čekanja [1]. Trenutna implementacija neposredno proverava prva dva razloga i validnost pripadnosti potpisnika grupi. Poseban tajmer za istek **IN_PREPARE2** faze nije implementiran.

Kada čvor utvrdi nepravilnost, poziva:

```

1 outPrepare.type = MessageType::OUT_PREPARE;
2 outPrepare.from = id_;
3 outPrepare.groupId = groupId_;

```

```

4 outPrepare.value = proposedValue_;
5 outPrepare.signatures.insert(id_);
6 outPrepare.voteWeight = 1;
7 outPrepare.nodeDecision = true;
8 network_.broadcastToOtherRepresentatives(outPrepare, groupId_);

```

Listing 3.1: Suština metode `sendNodeDecision`

Ovim se predstavnik problematične grupe zaobilazi. Ostali predstavnici mogu računati pojedinačne validne glasove običnih čvorova iz te grupe, osim ako kasnije ne prime validan normalni doprinos predstavnika iste grupe.

3.6 Threshold vote-counting model

Neka je F broj validnih potpisa u sertifikatu grupe, a P težina tog doprinosa. Simulator primenjuje pravilo:

$$P = \begin{cases} m, & F \geq m - E, \\ F, & F < m - E. \end{cases} \quad (3.15)$$

Za node-decision poruku običnog čvora važi $P = 1$. Ukupan zbir kod predstavnika je:

$$H = \sum_{g \in G_{normal}} P_g + \sum_{g \in G_{abnormal}, g \notin G_{normal}} |V_g|, \quad (3.16)$$

gde je V_g skup različitih običnih čvorova koji su poslali validan node-decision doprinos za grupu g .

Predstavnik šalje COMMIT kada važi:

$$H \geq (R - w)m. \quad (3.17)$$

3.7 Teorijska komunikaciona složenost

Rad daje aproksimaciju broja poruka u normalnom toku:

$$C = 2(n - 1) + 2(m - 1) \left\lfloor \frac{n - 1}{m} \right\rfloor + \left\lfloor \frac{n - 1}{m} \right\rfloor^2. \quad (3.18)$$

Dominantan član je R^2 , pa je složenost:

$$O\left(\left\lceil \frac{n-1}{m} \right\rceil^2\right). \quad (3.19)$$

Ova formula koristi pojednostavljen način brojanja iz rada. Simulator dodatno beleži klijentski zahtev, sve **REPLY** poruke, **COMMIT** poruke i samoglas predstavnika u **IN_PREPARE1**, zbog čega konkretan broj **SEND** događaja nije identičan vrednosti C . Detaljno poređenje dato je u poglavlju 10.

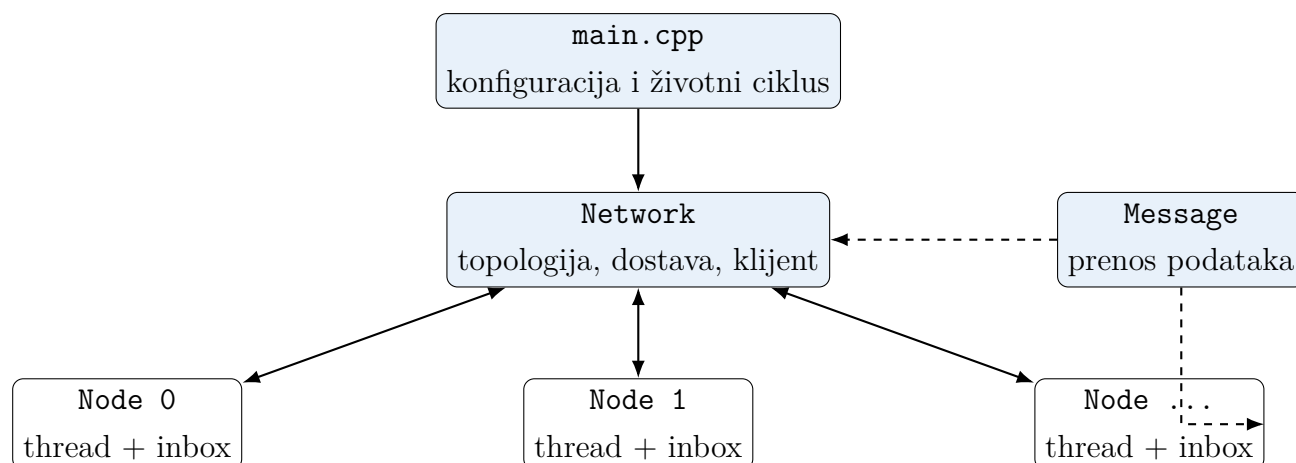
Poglavlje 4

Arhitektura simulatora

4.1 Pregled komponenti

Projekat je organizovan u četiri logičke komponente:

1. `Message.hpp` – model poruke i tipovi poruka;
2. `Node.hpp` / `Node.cpp` – stanje i algoritamska logika jednog čvora;
3. `Network.hpp` / `Network.cpp` – topologija, dostava poruka, pragovi, scenariji i klijent;
4. `main.cpp` – čitanje parametara, inicijalizacija, pokretanje i završni izveštaj.



Slika 4.1: Arhitektura simulatora u jednom procesu

4.2 Procesni model

Sve komponente rade u jednom operativnom procesu. To pojednostavljuje izradu i demonstraciju, ali čvorovi nisu obrađeni u jednoj centralnoj petlji. Svaki čvor ima svoju nit i sopstveni red poruka. `Network` ne poziva direktno algoritamski handler primaoca; umesto toga ubacuje poruku u njegov inbox. Primalac je obrađuje kada njegova nit dođe na red.

Ovakav dizajn ispunjava suštinu zahteva za konkurentnošću:

- redosled obrade poruka zavisi od raspoređivanja niti;
- svaki čvor može čekati nezavisno od ostalih;
- pristup inbox-u je zaštićen mutex-om;
- uspavana nit se budi preko uslovne promenljive.

4.3 Životni ciklus simulacije

Kompletan tok iz `main.cpp` je:

1. pročitati scenario, broj ispravnih čvorova, broj vizantijskih čvorova i veličinu grupe;
2. konstruisati `Network` sa ukupnim brojem čvorova;
3. pozvati `initialize()` i napraviti topologiju;
4. pozvati `configureScenario()` i dodeliti fault režime;
5. ispisati konfiguraciju;
6. pokrenuti sve niti metodom `start()`;
7. pokrenuti konsenzus za vrednost `BLOCK_1`;
8. zaustaviti niti;
9. ispisati konačno stanje svakog čvora i rezultat klijenta.

Poglavlje 5

Model poruke – Message.hpp

5.1 Tipovi poruka

Enum `MessageType` definiše osam tipova. Tabela 5.1 daje ulogu svakog tipa.

Tabela 5.1: Tipovi poruka

Tip	Namena
CLIENT_REQUEST	Klijent šalje novu vrednost primary čvoru.
PRE_PREPARE1	Primary objavljuje predlog svim replikama.
IN_PREPARE1	Čvor šalje lokalni glas predstavniku svoje grupe.
IN_PREPARE2	Predstavnik distribuira agregirani lokalni sertifikat članovima grupe.
OUT_PREPARE	Predstavnik šalje doprinos drugim predstavnicima; isti tip se koristi za node-decision poruke običnih čvorova.
COMMIT	Predstavnik koji je dostigao globalni prag šalje dokaz primary čvoru.
PRE_PREPARE2	Primary šalje završnu potvrdu svim replikama.
REPLY	Čvor šalje svoju konačnu odluku klijentu.

5.2 Polja strukture Message

Tabela 5.2: Polja poruke u trenutnoj implementaciji

Polje	Tip	Značenje
id	int	Jedinstveni redni broj koji dodeljuje mreža pri slanju.
type	MessageType	Vrsta protokolske poruke.
from	int	Identifikator pošiljaoca; klijent koristi <code>CLIENT_ID=-1</code> .
to	int	Identifikator primaoca ili klijenta.
view	int	Broj pogleda; u trenutnoj verziji ostaje 0.
sequenceNumber	int	Redni broj konsenzusne instance; po-drazumevano 1.
groupId	int	Grupa na koju se dokaz odnosi; <code>-1</code> kada nije primenljivo.
value	std::string	Predložena ili potvrđena vrednost.
signatures	std::set<int>	Simulirani potpisi predstavljeni jedinstvenim ID-jevima čvorova.
voteWeight	int	Težina doprinosa u threshold vote-counting modelu.
nodeDecision	bool	Obeležava abnormalni <code>OUT_PREPARE</code> koji šalje običan čvor.

5.3 Razlog korišćenja skupa potpisa

`std::set<int>` automatski uklanja duplikate, pa jedan pošiljalac ne može biti izbrojan više puta unutar istog objekta poruke. Provera `validGroupSignatures` dodatno potvrđuje da svaki ID pripada grupi za koju se sertifikat predstavlja.

Ovo nije kriptografski potpis. Ne dokazuje autentičnost, integritet niti posedovanje privatnog ključa. To je simulacioni dokaz koji omogućava proveru pravila kvoruma bez uključivanja kriptografske biblioteke.

Poglavlje 6

Klasa Network

6.1 Odgovornosti klase

`Network` je infrastrukturna komponenta simulatora. Njene glavne odgovornosti su:

- čuvanje globalnih parametara n , m , R , E i w ;
- validacija konfiguracije;
- formiranje heš-prstena i topologije grupa;
- kreiranje svih objekata `Node`;
- pokretanje i zaustavljanje niti;
- dodela grešaka prema scenariju;
- unicast i broadcast isporuka poruka;
- brojanje klijentskih odgovora;
- sinhronizovano logovanje.

6.2 Konstruktor i validacija parametara

Konstruktor izračunava sve pragove u inicijalizacionoj listi:

```
1 Network::Network(int numberOfNodes, int groupSize)
2     : numberOfNodes_(numberOfNodes),
3       groupSize_(groupSize),
4       E_((groupSize - 1) / 3),
5       R_((numberOfNodes - 1) / groupSize),
6       w_((R_ - 1) / 3),
```

```

7     localQuorum_(2 * E_ + 1),
8     globalThreshold_((R_ - w_) * groupSize),
9     replyThreshold_((numberOfNodes - 1) / 2 + 1) {
10    // validacija...
11 }

```

Listing 6.1: Izračunavanje NBFT parametara u konstruktoru

Validacija sprečava konfiguracije koje ne odgovaraju modelu rada:

- m mora biti najmanje 4 i oblika $3E + 1$;
- $n - 1$ mora biti deljivo sa m ;
- broj grupa mora biti najmanje 4.

Zbog ovih pravila, dozvoljeni primeri veličine grupe su $m = 4, 7, 10, \dots$. Za $m = 4$ najmanja potpuna mreža je $n = 17$.

6.3 Inicijalizacija topologije

Metoda `initialize()` najpre poziva `buildTopology()`, zatim kreira n čvorova i svakom prosleđuje:

- ID grupe ili -1 za primary;
- informaciju da li je čvor primary;
- informaciju da li je predstavnik grupe.

Nakon konfiguracije poziva se `reset()` kako bi svako lokalno stanje bilo prazno pre početka rada.

6.4 Formiranje heš-prstena

Metoda `buildHashRing()` generiše simulirane IP adrese, računa heš i sortira elemente. Struktura `RingEntry` sadrži samo `nodeId` i 32-bitni `hash`.

Metoda `clockwiseNode()` traži prvi element sa `entry.hash >= target`. Ako takav element ne postoji, bira prvi čvor u sortiranom vektoru, čime se realizuje kružno vraćanje.

Metoda `buildTopology()` zatim:

1. bira primary čvor;
2. izbacuje primary iz liste kandidata;

3. raspoređuje preostale čvorove u R grupa po m članova;
4. gradi lokalni heš-prsten svake grupe;
5. bira predstavnika svake grupe;
6. popunjava mapu `nodeToGroup_`.

6.5 Dostava poruka

Metoda `send(Message message)` je jedina centralna tačka isporuke. Ona dodeljuje poruci ID, upisuje događaj u log i zatim:

- ako je primalac klijent, poziva `receiveClientReply`;
- ako je ID primaoca nevažeći, odbacuje poruku uz log;
- u suprotnom poziva `nodes_[to]->enqueue(...)`.

Važno je da `send()` ne obrađuje algoritam u ime primaoca. Ona samo stavlja poruku u njegov red.

6.6 Vrste broadcast-a

Tabela 6.1: Broadcast metode klase Network

Metoda	Primaoci
<code>broadcastToReplicas</code>	Svi čvorovi osim primary čvora.
<code>broadcastToGroup</code>	Svi članovi određene grupe, uz opcioni izuzetak po ID-u.
<code>broadcastToRepresentatives</code>	Po jedan predstavnik svake grupe, uključujući pošiljaoca ako je i sam predstavnik.
<code>broadcastToOtherRepresentatives</code>	Predstavnici svih grupa osim navedene grupe; koristi se za node-decision.

6.7 Klijentska odluka

Klijent koristi mapu `clientReplies_` čiji je ključ vrednost, a sadržaj skup ID-jeva pošiljalaca. Time se odgovori za različite vrednosti ne sabiraju zajedno. Odgovor se prihvata samo ako sadrži tačno jedan potpis i taj potpis odgovara polju `from`.

Čekanje je implementirano metodom `wait_for` nad uslovnom promenljivom. Predikat je `clientDecided_`. Kada se dostigne prag, `receiveClientReply` postavlja odluku i poziva `notify_all()`.

6.8 Logovanje

Metoda `log()` koristi poseban mutex, pa se linije različitih niti ne mešaju. Svaki zapis dobija rastući broj događaja:

Listing 6.2: Primer konzolnog loga

```
[event=0009] CLIENT salje zahtev primary cvoru Node 14
[event=0010] SEND CLIENT_REQUEST | Node -1 -> Node 14
[event=0011] Primary Node 14 prihvata zahtev za BLOCK_1
[event=0012] SEND PRE_PREPARE1 | Node 14 -> Node 0
...
[event=0144] CLIENT POTVRDJUJE KONSENZUS ZA BLOCK_1
```

Broj događaja nije isto što i broj poruka, jer se loguju i konfiguracija, odluke i završna stanja.

Poglavlje 7

Klasa Node

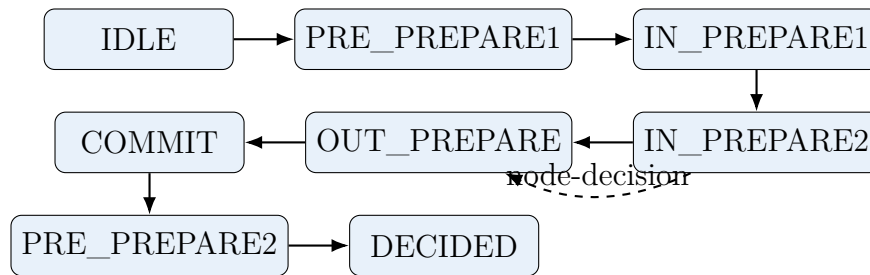
7.1 Lokalno stanje čvora

Svaki čvor čuva:

- identitet i referencu na mrežu;
- grupu i uloge primary/predstavnik;
- fault režim i trenutnu fazu;
- radnu nit, mutex, uslovnu promenljivu i red poruka;
- `view`, `sequenceNumber`, predloženu i odlučenu vrednost;
- lokalne glasove grupisane po vrednosti;
- zastavice koje sprečavaju duplo slanje sertifikata, commit-a i odgovora;
- normalne težine grupa i skupove abnormalnih pojedinačnih glasova.

7.2 Faze lokalnog automata

Enum `NodePhase` ima osam stanja: početno `IDLE`, šest protokolskih faza i završno `DECIDED`.



Slika 7.1: Pojednostavljen automat faza čvora

Nisu svi čvorovi nužno fizički prolazili kroz svako stanje na identičan način. Na primer, obična replika ne šalje COMMIT, ali na kraju prihvata PRE_PREPARE2. Enum služi za pregled i logovanje trenutne uloge u toku protokola.

7.3 Konkurentna petlja

Metoda `run()` izvršava sledeći obrazac:

```

1 while (true) {
2     Message message;
3     {
4         std::unique_lock<std::mutex> lock(inboxMutex_);
5         inboxReady_.wait(lock, [this] {
6             return !running_ || !inbox_.empty();
7         });
8         if (!running_ && inbox_.empty()) {
9             return;
10        }
11        message = std::move(inbox_.front());
12        inbox_.pop();
13    }
14    handle(message);
15 }
  
```

Listing 7.1: Petlja radne niti čvora

Mutex se otključava pre poziva `handle`. To je važno jer obrada poruke može izazvati slanje novih poruka, uključujući poruku istom čvoru. Da se mutex zadržava tokom obrade, povećao bi se rizik od blokiranja i nepotrebno bi se produžilo zaključavanje inbox-a.

7.4 Dispatcher poruka

Metoda `handle` najpre odbacuje sve zakašnjele poruke ako je čvor već u stanju `DECIDED`. Zatim `switch` prosleđuje poruku odgovarajućem handler-u. `REPLY` nema čvorovski handler jer ga obrađuje simulirani klijent u klasi `Network`.

7.5 Obrada pojedinačnih faza

7.5.1 `handleClientRequest`

Poruku obrađuje samo primary koji nije u režimu `SILENT`. Primary kopira view, sekvencu i vrednost, menja fazu i šalje `PRE_PREPARE1` svim replikama.

7.5.2 `handlePrePrepare1`

Replika odbacuje poruku ako je sama primary, ako je tiha ili ako pošiljalac nije trenutni primary. Nakon prihvatanja šalje `IN_PREPARE1` svom predstavniku. U režimu `WRONG_VALUE` menja sadržaj glasa, ali i dalje navodi sopstveni ID kao potpis.

7.5.3 `handleInPrepare1`

Ovu poruku obrađuje samo predstavnik odgovarajuće grupe. Proverava pripadnost pošiljaoca, veličinu potpisa i vezu potpisa sa pošiljaocem. Glas se zatim dodaje u skup za konkretnu vrednost.

Zastavica `localCertificateSent_` obezbeđuje da predstavnik formira najviše jedan lokalni sertifikat. Kada je kvorum dostignut, sertifikat se šalje ostalim članovima grupe i, ako i dalje ima dovoljan broj potpisa, predstavnik odmah pokreće normalni `OUT_PREPARE`.

U režimu `LOW_SIGNATURES` predstavnik namerno briše potpise dok broj ne postane manji od lokalnog kvoruma. Takav sertifikat ipak šalje grupi, kako bi obični čvorovi mogli da detektuju grešku.

7.5.4 `handleInPrepare2`

Običan ispravan čvor proverava vrednost, broj potpisa i pripadnost potpisnika. Ako je dokaz neispravan, poziva `sendNodeDecision`. Ispravan dokaz ne zahteva dodatnu poruku od običnog čvora jer predstavnik već šalje normalni `OUT_PREPARE`.

7.5.5 `sendNormalOutPrepare`

Metoda kopira lokalni sertifikat i računa težinu doprinosa. Za $m = 4$ i $E = 1$, svaki sertifikat sa najmanje tri potpisa dobija težinu četiri. Poruka se šalje svim predstavnicima.

7.5.6 `sendNodeDecision`

Metoda se može izvršiti samo jednom po čvoru zbog zastavice `nodeDecisionSent_`. Poruka sadrži jedan potpis i težinu jedan, a šalje se predstavnicima drugih grupa. Predstavnik sopstvene problematične grupe se namerno preskače.

7.5.7 `handleOutPrepare`

Poruku obrađuje samo ispravan predstavnik. Postoje dve grane:

- **node-decision doprinos:** mora imati jedan potpis, pošiljalac mora biti član navedene grupe i ne sme biti predstavnik te grupe;
- **normalni doprinos:** pošiljalac mora biti stvarni predstavnik grupe, broj potpisa mora biti najmanje lokalni kvorum, a svi potpisnici moraju pripadati grupi.

Ako stigne validan normalni doprinos, eventualni pojedinačni abnormalni glasovi iste grupe se brišu, kako ista grupa ne bi bila dvostruko uračunata.

7.5.8 `trySendCommit`

Metoda sabira težine i proverava globalni prag. Zastavica `commitSent_` sprečava duplo slanje. COMMIT sadrži identitet predstavnika, grupu, vrednost, njegov potpis i ukupnu težinu H .

7.5.9 `handleCommit`

Poruku prihvata samo ispravan primary. Pošiljalac mora biti neki predstavnik, vrednost mora odgovarati predlogu, a težina mora dostići globalni prag. Prvi validni dokaz pokreće PRE_PREPARE2 broadcast. Primary zatim i sam donosi odluku i šalje REPLY.

7.5.10 `handlePrePrepare2` i `decideAndReply`

Ispravna replika proverava primary, težinu i usklađenost vrednosti. Funkcija `decideAndReply` čuva odluku, prelazi u DECIDED i šalje tačno jedan odgovor klijentu. Vizantijski čvorovi u bilo kom nepoštenom režimu ne prihvataju završnu poruku, jer handler zahteva `FaultMode::HONEST`.

Poglavlje 8

Scenariji grešaka

8.1 FaultMode

Enum `FaultMode` sadrži četiri režima:

Tabela 8.1: Režimi ponašanja čvora

Režim	Ponašanje
HONEST	Čvor prati pravila protokola.
WRONG_VALUE	U lokalnom glasu šalje vrednost <code>BLOCK_1_WRONG</code> .
LOW_SIGNATURES	Predstavnik namerno šalje sertifikat sa manje od $2E+1$ potpisa.
SILENT	Čvor ne reaguje na poruke koje bi trebalo da obradi. U dostupnom scenariju ovaj režim se dodeljuje primary čvoru.

8.2 Konfigurabilni scenariji

8.2.1 normal

Zahteva nula vizantijskih čvorova. Svi čvorovi su `HONEST`. Očekuje se da svi čvorovi odluče `BLOCK_1`, a klijent dobije najmanje devet odgovora.

8.2.2 byzantine_wrong_value

Broj navedenih vizantijskih čvorova bira se među običnim članovima grupa, ne među predstavnicima. Kandidati se obilaze po poziciji unutar grupa, pa se greške raspoređuju preko više grupa. Vizantijski čvorovi šalju pogrešnu vrednost u `IN_PREPARE1` fazi.

Za konfiguraciju 13 ispravnih i 4 vizantijska čvora, po jedan običan čvor u svakoj grupi

šalje pogrešan glas. Preostala tri ispravna glasa po grupi i dalje zadovoljavaju lokalni kvorum.

8.2.3 `faulty_rep_low`

Predstavnik prve grupe dobija režim `LOW_SIGNATURES`. Ako je zadato više od jednog vizantijskog čvora, ostali se biraju među običnim čvorovima i dobijaju režim `WRONG_VALUE`.

Članovi prve grupe detektuju premali sertifikat i šalju node-decision poruke drugim predstavnicima. U konfiguraciji sa jednom greškom, tri preostale normalne grupe zajedno nose težinu $3 \cdot 4 = 12$, što je tačno globalni prag. Zato mreža može završiti konsenzus i bez normalnog doprinosa prve grupe.

8.2.4 `primary_silent`

Primary dobija režim `SILENT`. Klijentski zahtev stiže u njegov inbox, ali handler ne pokreće `PRE_PREPARE1`. Pošto trenutna verzija nema promenu view-a, klijent čeka 2500 ms i prijavljuje neuspeh.

8.3 Izbor neispravnih čvorova

Pomoćna metoda `ordinaryNodeCandidates()` obilazi istu poziciju kroz sve grupe, pa prelazi na sledeću poziciju. Predstavnici se preskaču. Ovaj izbor je deterministički za istu topologiju, što olakšava ponavljanje demonstracije.

Poglavlje 9

Pokretanje i korišćenje

9.1 Struktura datoteka

Očekivana struktura projekta je:

```
project/
|-- README.md
|-- src/
|   |-- main.cpp
|   |-- Message.hpp
|   |-- Network.hpp
|   |-- Network.cpp
|   |-- Node.hpp
|   '-- Node.cpp
'-- consensus_simple.exe
```

9.2 Kompajliranje

Za MinGW/G++ u Windows PowerShell terminalu koristi se:

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic -pthread \
    src/main.cpp src/Node.cpp src/Network.cpp \
    -o consensus_simple.exe
```

Opcija `-std=c++17` je neophodna zbog korišćenih C++ mogućnosti, a `-pthread` uključuje podršku za niti.

9.3 Argumenti komandne linije

Program očekuje tačno četiri argumenta:


```
consensus_simple.exe <scenario> <ispravni> <Byzantine> <velicina_grupe>
```

Ukupan broj čvorova računa se kao:

$$n = \text{ispravni} + \text{Byzantine}. \quad (9.1)$$

Primeri:

```
.\consensus_simple.exe normal 17 0 4  
.\consensus_simple.exe byzantine_wrong_value 13 4 4  
.\consensus_simple.exe faulty_rep_low 16 1 4  
.\consensus_simple.exe primary_silent 16 1 4
```

9.4 Validacija ulaza

Program prijavljuje grešku kada:

- broj argumenata nije četiri;
- broj ispravnih ili vizantijskih čvorova je negativan;
- scenario nije poznat;
- scenario **normal** ima vizantijske čvorove;
- ne-normalni scenario nema nijedan vizantijski čvor;
- nema dovoljno običnih čvorova za dodelu greške;
- m nije oblika $3E + 1$;
- $n - 1$ nije deljivo sa m ;
- postoji manje od četiri grupe.

Poglavlje 10

Testiranje i rezultati

10.1 Metod kontrolne provere

Dostavljeni izvorni kod je kontrolno preveden sa G++ u C++17 režimu, sa uključenim upozorenjima i podrškom za niti. Pokrenuta su sva četiri scenarija navedena u `README.md`. Za svaki scenario zabeležen je konačni rezultat, broj čvorova u stanju `DECIDED`, broj node-decision događaja i broj `SEND` zapisa po tipu poruke.

Ovi rezultati predstavljaju proveru konkretne verzije izvornog koda. Redosled pojedinačnih događaja može blago varirati zbog raspoređivanja niti, ali pragovi i konačan ishod treba da ostanu isti.

10.2 Rezime ishoda

Tabela 10.1: Rezultati kontrolnog pokretanja

Scenario	Konsenzus	DECIDED	Node-decision	Objašnjenje
normal 17 0 4	da	17	0	Svi čvorovi prihvataju <code>BLOCK_1</code> .
byzantine_wrong_value 13 4 4	da	13	0	Četiri pogrešna glasa ne sprečavaju lokalni kvorum od tri ispravna glasa po grupi.
faulty_rep_low 16 1 4	da	16	3	Tri obična člana problematične grupe detektuju premali sertifikat i zaobilaze predstavnika.
primary_silent 16 1 4	ne	0	0	Primary ne pokreće protokol, a view change nije implementiran.

10.3 Broj poslatih poruka

Tabela 10.2: Broj SEND događaja po scenariju

	Req	PP1	IP1	IP2	OP	Com	PP2	Reply	Ukupno
normal	1	16	16	12	16	4	16	17	98
wrong value	1	16	16	12	16	4	16	13	94
faulty rep low	1	16	16	12	21	3	16	16	101
primary silent	1	0	0	0	0	0	0	0	1

Skraćenice u tabeli označavaju redom: CLIENT_REQUEST, PRE_PREPARE1, IN_PREPARE1, IN_PREPARE2, OUT_PREPARE, COMMIT, PRE_PREPARE2 i REPLY.

10.3.1 Zašto normalni tok ima 98 poruka

Za $n = 17$, $m = 4$ i $R = 4$ broj poruka u ovoj implementaciji može se direktno razložiti:

$$1 \text{ klijentski zahtev,} \quad (10.1)$$

$$16 \text{ prvi broadcast primary čvora,} \quad (10.2)$$

$$16 \text{ lokalni glasovi, uključujući glas predstavnika samom sebi,} \quad (10.3)$$

$$12 \text{ lokalni sertifikati ka tri ostala člana svake grupe,} \quad (10.4)$$

$$16 \text{ četiri predstavnika ka četiri predstavnika,} \quad (10.5)$$

$$4 \text{ po jedan COMMIT svakog predstavnika,} \quad (10.6)$$

$$16 \text{ završni broadcast primary čvora,} \quad (10.7)$$

$$17 \text{ odgovori svih čvorova klijentu.} \quad (10.8)$$

Zbir je 98. Teorijska formula rada za istu konfiguraciju daje:

$$C = 2 \cdot 16 + 2 \cdot 3 \cdot 4 + 4^2 = 72. \quad (10.9)$$

Razlika od 26 poruka nastaje zato što simulator dodatno broji četiri samoglasa predstavnika u lokalnoj fazi, četiri COMMIT poruke, jedan zahtev klijenta i sedamnaest odgovora klijentu:

$$72 + 4 + 4 + 1 + 17 = 98. \quad (10.10)$$

Ovo nije kontradikcija, već različita konvencija brojanja i konkretan detalj simulatora.

10.4 Analiza scenarija sa pogrešnom vrednošću

U ovom scenariju četiri vizantijska obična čvora ne donose odluku, pa se šalje samo trinaest `REPLY` poruka. Klijentski prag je devet, tako da je trinaest odgovora dovoljno. Predstavnici vode glasove po vrednosti, pa jedan pogrešan glas ne kontaminira sertifikat za `BLOCK_1`.

10.5 Analiza neispravnog predstavnika

Predstavnik prve grupe šalje sertifikat sa dva potpisa umesto tri. Tri ostala člana grupe šalju po jednu `node-decision` poruku svakom od tri predstavnika drugih grupa, što daje $3 \cdot 3 = 9$ dodatnih `OUT_PREPARE` poruka. Normalni doprinosi tri ispravne grupe daju 12 poruka, pa je ukupan broj `OUT_PREPARE` poruka 21.

Samo tri ispravna predstavnika šalju `COMMIT`; neispravan predstavnik ne učestvuje dalje. Ipak, zbir tri potpuno validne grupe je 12, što zadovoljava globalni prag.

10.6 Primer završnog stanja

U normalnom scenariju završni redovi imaju oblik:

```
[event=0167] Node 13 | grupa=0 | faza=DECIDED | fault=HONEST | odluka=BLOCK_1
[event=0168] Node 14 | grupa=-1 | faza=DECIDED | fault=HONEST | odluka=BLOCK_1
[event=0169] Node 15 | grupa=0 | faza=DECIDED | fault=HONEST | odluka=BLOCK_1
[event=0170] Node 16 | grupa=0 | faza=DECIDED | fault=HONEST | odluka=BLOCK_1
```

NBFT konsenzus je uspesno završen.

Vrednost `grupa=-1` označava primary čvor, koji nije član nijedne grupe.

Poglavlje 11

Bezbednosne provere u implementaciji

11.1 Provere pošiljaoca i uloge

Handler-i ne veruju automatski polju `from`. Za važne poruke proverava se da li je pošiljalac:

- trenutni `primary` za `PRE_PREPARE1` i `PRE_PREPARE2`;
- član odgovarajuće grupe za `IN_PREPARE1`;
- predstavnik odgovarajuće grupe za normalni `OUT_PREPARE`;
- bilo koji predstavnik za `COMMIT`;
- običan član, a ne predstavnik, za `node-decision` doprinos.

11.2 Provere potpisa

Iako su potpisi simulirani, implementacija proverava:

- da običan glas ima tačno jedan potpis;
- da je potpis jednak ID-u pošiljaoca;
- da svi potpisnici lokalnog sertifikata pripadaju navedenoj grupi;
- da sertifikat ima najmanje lokalni kvorum;
- da klijentski odgovor ima tačno jedan potpis pošiljaoca.

11.3 Sprečavanje višestrukog brojanja

Korišćenje skupova sprečava ponavljanje istog ID-a. Zastavice `localCertificateSent_`, `nodeDecisionSent_`, `commitSent_`, `finalBroadcastSent_` i `replySent_` sprečavaju da čvor više puta izvrši istu ključnu akciju.

Normalni doprinos grupe ima prednost nad ranije primljenim pojedinačnim node-decision glasovima iste grupe. Time se ista grupa ne računa istovremeno kao puna grupa i kao zbir pojedinačnih čvorova.

11.4 Razdvajanje glasova po vrednosti

Lokalni glasovi i klijentski odgovori čuvaju se u mapama čiji je ključ vrednost. Ova odluka je bitna jer sprečava da se glasovi za `BLOCK_1` i `BLOCK_1_WRONG` saberu u isti kvorum.

Poglavlje 12

Pojednostavljenja i ograničenja

12.1 Simulirani, a ne kriptografski potpisi

Skup ID-jeva nije zaštita od falsifikovanja. U stvarnom sistemu poruka bi sadržala digitalni potpis nad digestom vrednosti, brojem view-a, sekvencom i drugim kontekstom. Primalac bi proveravao javni ključ svakog potpisnika.

12.2 Jedan proces i memorijska komunikacija

Čvorovi jesu konkurentni, ali nisu odvojeni procesi niti fizičke mašine. `Network::send` koristi poziv metode i ubacivanje objekta u memorijski red. Nema TCP/UDP soketa, serijalizacije, mrežnih particija niti gubitka paketa.

12.3 Statičan view

Polje `view` postoji i učestvuje u izboru primary čvora, ali trenutna aplikacija izvršava jednu instancu sa `view=0`. Ne postoji `VIEW_CHANGE` poruka, izbor novog primary čvora posle timeout-a niti prenos dokaza u novi view.

12.4 Jedna konsenzusna instanca

Program pokreće konsenzus samo za `BLOCK_1` i koristi `sequenceNumber=1`. Nema petlje za više blokova, nema trajnog loga odluka i nema zaštite od mešanja poruka više paralelnih instanci izvan jednostavnih polja `view/sequence/value`.

12.5 Nepotpuna timeout logika node-decision modela

Rad predviđa da običan čvor aktivira node-decision i kada ne dobije `IN_PREPARE2` poruku u ograničenom roku. Trenutni kod reaguje samo na primljeni, ali neispravan sertifikat. Zato potpuno tih predstavnika grupe ne bi automatski bio zaobiđen bez dodatnog tajmera.

12.6 Pojednostavljen dokaz COMMIT poruke

COMMIT prenosi ukupnu težinu i potpis predstavnika koji ga šalje, ali ne prenosi kompletan skup pojedinačnih grupnih sertifikata. Primary zato proverava težinu i ulogu pošiljaoca, ali ne može nezavisno rekonstruisati ceo zbir H iz same poruke.

12.7 Nema eksplicitne detekcije ekvivokacije

Mapa glasova po vrednosti sprečava mešanje vrednosti, ali nema posebne evidencije da je isti čvor pokušao da glasa za dve vrednosti u istoj instanci. `std::set` uklanja duplikate samo unutar jednog skupa.

12.8 Log samo na standardnom izlazu

Trenutna verzija numerise i prikazuje događaje na konzoli. Ne upisuje automatski izvršavanje u zaseban fajl i ne vodi trajne agregatne metrike. Rezultati se mogu sačuvati preusmeravanjem izlaza operativnog sistema.

Poglavlje 13

Predlozi za dalje unapređenje

Sledeći koraci prirodno proširuju postojeću arhitekturu, bez menjanja osnovne podele na Message, Node i Network:

1. **View change.** Dodati VIEW_CHANGE poruku, timeout primary čvora, novi izbor topologije i reset stanja za novi view.
2. **Timeout za IN_PREPARE2.** Svaki običan čvor može pokrenuti tajmer nakon PRE_PREPARE1; ako sertifikat ne stigne, šalje node-decision.
3. **Strukturirani dokazi.** Uvesti posebne strukture za lokalni sertifikat, doprinos grupe i globalni sertifikat, umesto opštih polja `signatures` i `voteWeight`.
4. **Vezivanje potpisa za instancu.** Potpis treba da pokriva view, sekvencu, identitet zahteva i digest vrednosti.
5. **Kašnjenje i gubitak poruka.** U mrežu dodati raspoređivač dostave, nasumično kašnjenje, odbacivanje i promenu redosleda poruka.
6. **Više uzastopnih blokova.** Nakon odluke uvećati sekvencu, ažurirati `previousHash` i ponovo izgraditi topologiju.
7. **Metrike.** Brojati poslate, isporučene i odbačene poruke, meriti trajanje faza i automatski izvoziti CSV rezultate.
8. **Statistički eksperiment.** Pokrenuti više stotina simulacija za različit broj vizantijskih čvorova, kao u FND analizi rada.
9. **Prava mrežna komunikacija.** Čvorove razdvojiti u procese ili kontejnere i koristiti sokete ili message broker.
10. **Automatski testovi.** Dodati unit testove za pragove, topologiju, validaciju

potpisa i scenarije grešaka.

Poglavlje 14

Zaključak

U projektu je implementiran konkurentni simulator NBFT konsenzusa u jeziku C++17. Osnovna ideja rada preneti je u program kroz podelu mreže na grupe, izbor primary čvora i predstavnika pomoću heš-prstena, lokalni konsenzus, međugrupno glasanje, node-decision broadcast i threshold vote-counting.

Arhitektura jasno razdvaja podatke poruke, lokalnu logiku čvora i mrežnu infrastrukturu. Svaki čvor ima sopstvenu nit i inbox, pa protokol nije realizovan kao sekvencijalna centralna petlja. Parametri i pragovi računaju se direktno iz n i m , a konstruktor sprečava konfiguracije koje nisu u skladu sa pretpostavkama modela.

Kontrolna testiranja pokazuju da normalan scenario završava konsenzusom, da četiri obična vizantijska čvora mogu biti tolerisana kada svaka grupa i dalje ima lokalni kvorum, i da node-decision mehanizam omogućava konsenzus uprkos predstavniku koji šalje premalo potpisa. Scenario tihog primary čvora pokazuje granicu trenutne verzije: bez view-change mehanizma nema oporavka od neodgovarajućeg lidera.

Simulator zato uspešno ispunjava osnovni obrazovni cilj projekta: omogućava praćenje stvarnog toka poruka i odluka u NBFT protokolu, uključujući relevantne nepravilnosti, uz jasno izdvojena pojednostavljenja koja ne treba mešati sa funkcionalnostima potpunog produkcionog sistema.

Dodatak A

Pregled javnih metoda

A.1 Network

Metoda	Namena
<code>initialize()</code>	Gradi topologiju, kreira čvorove i dodeljuje uloge.
<code>configureScenario(...)</code>	Resetuje fault režime i postavlja izabrani scenario.
<code>start() / stop()</code>	Pokreće ili zaustavlja sve niti čvorova.
<code>runConsensus(...)</code>	Resetuje klijenta, šalje zahtev i čeka odluku do timeout-a.
<code>send(...)</code>	Dodeljuje ID, loguje i isporučuje jednu poruku.
<code>broadcastToReplicas(...)</code>	Šalje poruku svim čvorovima osim primary čvora.
<code>broadcastToGroup(...)</code>	Šalje poruku članovima jedne grupe.
<code>broadcastToRepresentatives(...)</code>	Šalje poruku svim predstavnicima.
<code>broadcastToOtherRepresentatives(...)</code>	Šalje poruku predstavnicima drugih grupa.
<code>representativeForGroup(...)</code>	Vraća ID predstavnika grupe.
<code>nodeBelongsToGroup(...)</code>	Proverava članstvo čvora u grupi.
<code>isRepresentative(...)</code>	Proverava da li je čvor predstavnik bilo koje grupe.
<code>log(...)</code>	Sinhronizovano ispisuje numerisani događaj.
<code>printConfiguration()</code>	Ispisuje parametre, primary i sastav grupa.
<code>printFinalState()</code>	Ispisuje fazu, grešku i odluku svakog čvora.

A.2 Node

Metoda	Namena
<code>configureTopology(...)</code>	Dodeljuje grupu i uloge čvora.
<code>setFaultMode(...)</code>	Postavlja simulirano neispravno ponašanje.
<code>reset()</code>	Prazni inbox i briše stanje konsenzusa.
<code>start() / stop()</code>	Upravlja radnom niti čvora.
<code>enqueue(...)</code>	Bezbedno dodaje poruku u inbox i budi nit.
<code>id(), groupId()</code>	Vraćaju identitet i grupu.
<code>isPrimary(),</code>	Vraćaju trenutne uloge.
<code>isRepresentative()</code>	
<code>faultMode(), phase()</code>	Vraćaju fault režim i fazu.
<code>decidedValue()</code>	Vraća konačnu vrednost ili prazan string.

Dodatak B

Pseudokod kompletne simulacije

```
ULAZ: scenario, honestNodes, byzantineNodes, groupSize

n <- honestNodes + byzantineNodes
proveri m = 3E + 1, n = Rm + 1 i R >= 4
izgradi hash prsten
izaberi primary
podeli n-1 replika u R grupa
izaberi predstavnika svake grupe
kreiraj po jednu nit i inbox za svaki cvor
postavi fault rezime prema scenariju

CLIENT -> PRIMARY: CLIENT_REQUEST(value)

ako PRIMARY nije SILENT:
    PRIMARY -> sve replike: PRE_PREPARE1(value)

za svaku repliku paralelno:
    replika -> predstavnik grupe: IN_PREPARE1(value, potpis)

za svakog predstavnika paralelno:
    kada primi najmanje 2E+1 glasova za istu vrednost:
        formira IN_PREPARE2 sertifikat
        salje ga clanovima grupe
        ako je sertifikat validan:
            salje OUT_PREPARE svim predstavnicima

za svaki obican cvor:
    ako IN_PREPARE2 nije validan:
        salje node-decision OUT_PREPARE drugim predstavnicima

za svakog ispravnog predstavnika:
    proverava normalne i pojedinačne doprinose
```

```
racuna ukupnu tezinu H
ako  $H \geq (R-w)m$ :
    salje COMMIT primary cvoru

PRIMARY prihvata prvi validan COMMIT:
    PRIMARY -> sve replike: PRE_PREPARE2(value, H)
    PRIMARY odlucuje i salje REPLY

svaka ispravna replika koja prihvati PRE_PREPARE2:
    odlucuje value
    salje REPLY klijentu

CLIENT grupise odgovore po vrednosti
ako broj razlicitih posiljalaca za istu vrednost  $\geq (n-1)/2+1$ :
    konsenzus je uspesan
inace, po isteku timeout-a:
    konsenzus nije postignut
```

Dodatak C

Kontrolna lista za demonstraciju

Pre odbrane projekta korisno je proveriti sledeće:

1. Kod se prevodi bez grešaka komandom iz README fajla.
2. Normalni scenario ispisuje četiri grupe, primary i predstavnike.
3. U logu se mogu prepoznati svih šest NBFT faza.
4. Normalni scenario završava odlukom svih 17 čvorova.
5. Scenario sa pogrešnom vrednošću pokazuje da se glasovi razdvajaju po vrednosti.
6. Scenario `faulty_rep_low` ispisuje tri node-decision događaja.
7. Scenario `primary_silent` završava timeout-om i koristi se da se objasni nedostatak view-change mehanizma.
8. Objasniti razliku između simuliranog skupa ID-jeva i stvarnog digitalnog potpisa.
9. Objasniti lokalni prag $2E + 1$, globalni prag $(R - w)m$ i klijentski prag $(n - 1)/2 + 1$.
10. Naglasiti da **Network** samo isporučuje poruke, dok algoritamsku odluku donosi svaki **Node** u svojoj niti.

Literatura

- [1] J. Yang, Z. Jia, R. Su, X. Wu and J. Qin, “Improved Fault-Tolerant Consensus Based on the PBFT Algorithm,” *IEEE Access*, vol. 10, pp. 30274–30283, 2022, doi: 10.1109/ACCESS.2022.3153701.
- [2] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation*, pp. 173–186, 1999.
- [3] L. Lamport, R. Shostak and M. Pease, “The Byzantine Generals Problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.
- [4] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine and D. Lewin, “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web,” *Proceedings of the 29th Annual ACM Symposium on Theory of Computing*, pp. 654–663, 1997.
- [5] Specifikacija projekta iz predmeta *Distribuirani algoritmi*, interni nastavni materijal, 2026.